

PHYS 449-HW8

*ChatGPT was used to help with the assignment

$$1- C = \frac{\partial \langle E \rangle}{\partial T} \quad \langle E \rangle = \sum_x H(x) p(x) = \frac{1}{Z} \sum_x H(x) e^{-H(x)/T}$$

$$C = \frac{\partial \langle E \rangle}{\partial T} = \frac{1}{Z} \sum_x H \frac{\partial e^{-H/T}}{\partial T} - \frac{1}{Z^2} \frac{\partial Z}{\partial T} \sum_x H e^{-H/T}$$

$$= \frac{1}{Z} \sum_x H e^{-H/T} \left(\frac{H}{T^2} \right) - \frac{1}{Z^2} \left(\sum_x e^{-H/T} \frac{H}{T^2} \right) \left(\sum_x H e^{-H/T} \right)$$

$$= \frac{1}{T^2} \left(\frac{1}{Z} \sum_x H^2 e^{-H/T} - \left(\frac{1}{Z} \sum_x e^{-H/T} H \right)^2 \right)$$

$$= \frac{1}{T^2} \left(\langle E^2 \rangle - \langle E \rangle^2 \right)$$

PHYS 449 - Homework 8, Question 2

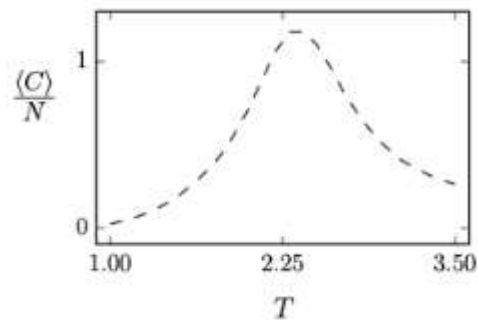
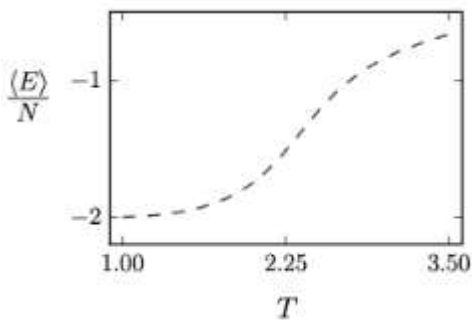
Student Name: Allen Mathew

Student ID: 20964892

Due Date: Thursday, November 27, 3:00pm

ChatGPT was used to help generate the code

2. Modify your Monte Carlo code from Homework 1 to compute the internal energy E and the specific heat C as a function of temperature T . Reproduce the below plot for a square lattice of size $N = 16 \times 16 = 256$. Pick a reasonable number of temperature points T_i , and collect a ballpark of 105 samples at each T_i after warmup/equilibration. Use error bars to indicate one standard deviation of uncertainty in the mean values.



```
In [5]: import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm

# ----- FAST vectorized 2D Ising Metropolis (checkerboard) -----
def init_spins(L, rng):
    return rng.choice(np.array([-1, 1], dtype=np.int8), size=(L, L))

def total_energy(spins, J=1.0):
    # count each bond once: right + down
    E = (spins * np.roll(spins, -1, axis=0)).sum()
    E += (spins * np.roll(spins, -1, axis=1)).sum()
    return -J * E

def checkerboard_masks(L):
    ii, jj = np.indices((L, L))
    even = ((ii + jj) & 1) == 0
    odd = ~even
    return even, odd

def metropolis_sweep_checkerboard(spins, T, J, rng, even_mask, odd_mask):
    # Update even sites simultaneously, then odd sites.
    for mask in (even_mask, odd_mask):
```

```

nn = (np.roll(spins, 1, 0) + np.roll(spins, -1, 0) +
      np.roll(spins, 1, 1) + np.roll(spins, -1, 1))
dE = 2.0 * J * spins * nn # energy change if flipped

# acceptance: always accept if dE<=0; else accept with exp(-dE/T)
accept = (dE <= 0.0)
r = rng.random(spins.shape)
accept |= (r < np.exp(-dE / T))

flip = mask & accept
spins[flip] *= -1

def run_chain_energies(L, T, J, warmup_sweeps, n_samples, sweeps_between_samples, seed):
    rng = np.random.default_rng(seed)
    spins = init_spins(L, rng)
    even, odd = checkerboard_masks(L)

    # warmup
    for _ in range(warmup_sweeps):
        metropolis_sweep_checkerboard(spins, T, J, rng, even, odd)

    energies = np.empty(n_samples, dtype=np.float64)
    for i in range(n_samples):
        for _ in range(sweeps_between_samples):
            metropolis_sweep_checkerboard(spins, T, J, rng, even, odd)
            energies[i] = total_energy(spins, J)
    return energies

# ----- fast 1σ uncertainties via blocking -----
def blocking_stats_E_c(energies, T, Nspins, block_size=200):
    M = len(energies)
    B = M // block_size
    if B < 3:
        raise ValueError("Need >=3 blocks: increase n_samples or reduce block_size.")

    e = energies[:B*block_size].reshape(B, block_size)

    # block estimates
    Eblk = e.mean(axis=1) / Nspins
    E2blk = (e**2).mean(axis=1)
    Emean_raw = e.mean(axis=1) # per-block <E> (not per spin)
    cblk = (E2blk - Emean_raw**2) / (Nspins * T**2)

    # overall means
    E_mean = Eblk.mean()
    c_mean = cblk.mean()

    # 1σ uncertainty of the mean (standard error across blocks)
    E_err = Eblk.std(ddof=1) / np.sqrt(B)
    c_err = cblk.std(ddof=1) / np.sqrt(B)

    return E_mean, E_err, c_mean, c_err

# ----- Q2 run -----
L = 16
J = 1.0

```

```

Nspins = L * L

Ts = np.linspace(1.0, 4.0, 25)
Tc = 2.269 * J

warmup_sweeps = 1000
n_samples = 20000
sweeps_between_samples = 1
block_size = 200

Emean = np.empty_like(Ts)
Eerr = np.empty_like(Ts)
cmean = np.empty_like(Ts)
cerr = np.empty_like(Ts)

for i, T in enumerate(tqdm(Ts, desc="Q2 temperatures")):
    energies = run_chain_energies(
        L=L, T=T, J=J,
        warmup_sweeps=warmup_sweeps,
        n_samples=n_samples,
        sweeps_between_samples=sweeps_between_samples,
        seed=1234+i
    )
    Emean[i], Eerr[i], cmean[i], cerr[i] = blocking_stats_E_c(
        energies, T=T, Nspins=Nspins, block_size=block_size
    )

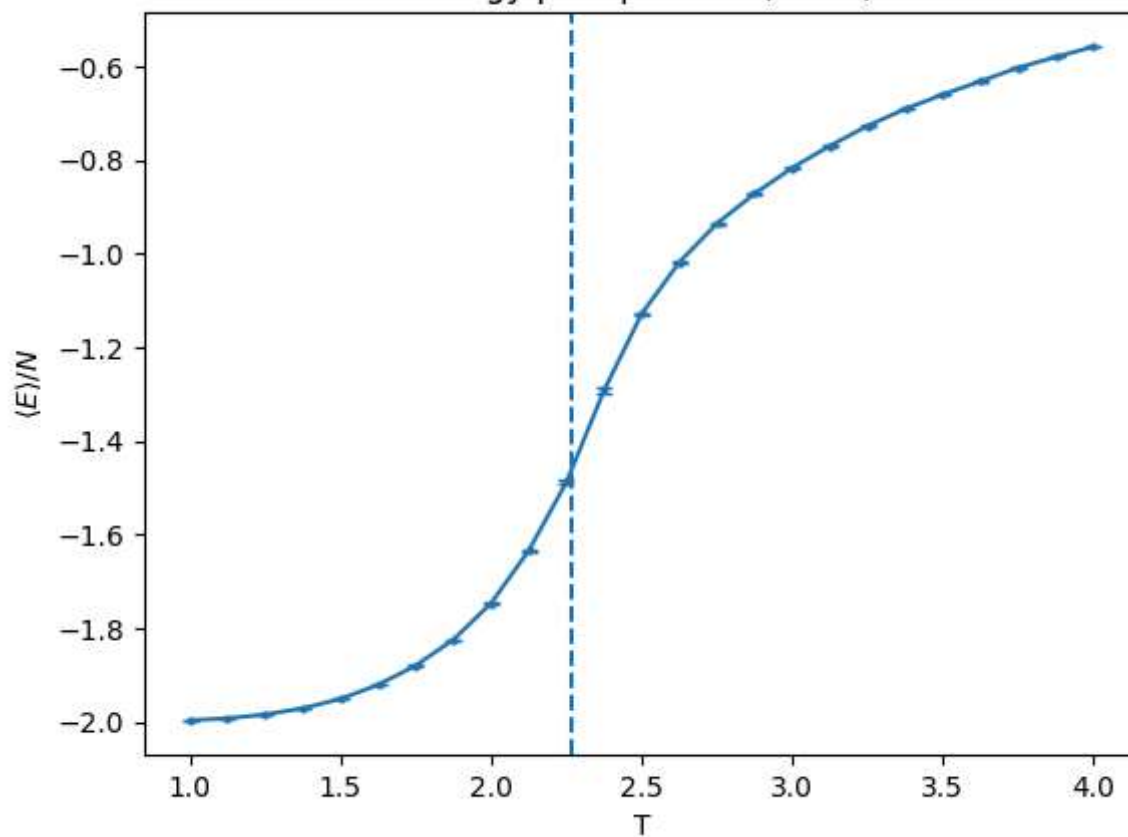
plt.figure()
plt.errorbar(Ts, Emean, yerr=Eerr, fmt="o-", ms=2, capsize=3)
plt.axvline(Tc, linestyle="--")
plt.xlabel("T")
plt.ylabel(r"$\langle E \rangle / N$")
plt.title(f"Energy per spin vs T (L={L})")
plt.show()

plt.figure()
plt.errorbar(Ts, cmean, yerr=cerr, fmt="o-", ms=2, capsize=3)
plt.axvline(Tc, linestyle="--")
plt.xlabel("T")
plt.ylabel(r"$c = C / N$")
plt.title(f"Specific heat per spin vs T (L={L})")
plt.show()

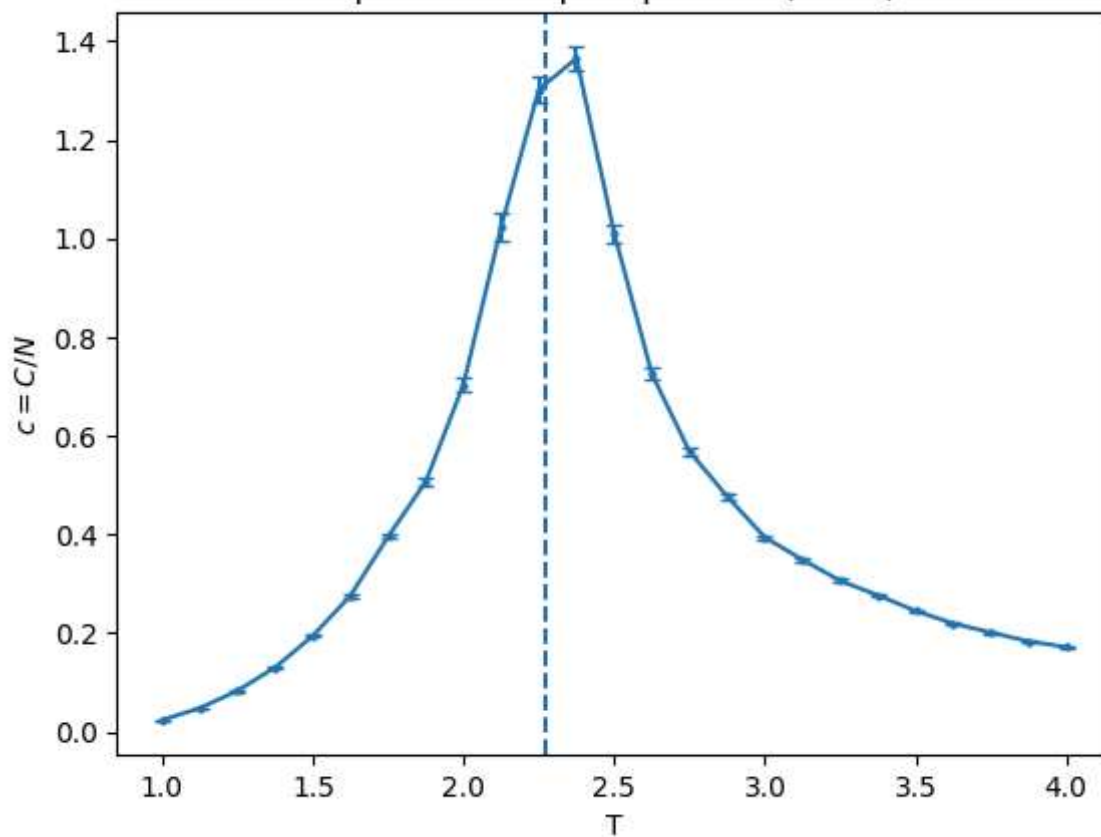
```

Q2 temperatures: 100% | | 25/25 [01:08<00:00, 2.73s/it]

Energy per spin vs T (L=16)



Specific heat per spin vs T (L=16)



The plots are similar to the plots shown in the question, they share the same overall pattern. By looking at the error bars, we can see that they increase as you get closer to the critical temperature. This makes sense because near the critical temperature, the system develops more fluctuations. This increases the uncertainty in both $\langle E \rangle$ and $\langle C \rangle$.

PHYS 449 - Homework 8, Question 3

Student Name: Allen Mathew

Student ID: 20964892

Due Date: Thursday, November 27, 3:00pm

ChatGPT was used to help generate the code

3. Train three Restricted Boltzmann Machines (RBMs) on the spin configurations generated in Question 2, at $T_1 = 1.00$, $T_2 = 2.27$ (the critical temperature), and $T_3 = 3.50$. Start with 4 hidden units for the RBM, a learning rate of 10^{-2} or less, and train with contrastive divergence CD-k. Try $k < 10$, with a minibatch size of 100 or less, and aim for say 103 epochs of training. After you are satisfied with the training, generate new spin configurations randomly from each trained RBM. Use these new spin configurations and compute $\langle E \rangle$ and $\langle C \rangle$. Compare the results with those obtained in Question 2, and comment on the source of any discrepancies.

```
In [4]: import numpy as np
import torch
import pandas as pd
import MonteCarlo as mc # uses your HW1 Ising code: init_spins, total_energy, ener
from tqdm import tqdm

# =====
# 0) Reproducibility
# =====
SEED = 123
np.random.seed(SEED)
torch.manual_seed(SEED)

device = "cuda" if torch.cuda.is_available() else "cpu"
print("device:", device)

# =====
# 1) Generate MC configs at T1,T2,T3 (Q2-style data)
# We add explicit warmup/equilibration (not in your run_mcmc).
# =====
def run_mcmc_with_warmup(L, n_samples, T, J=1.0, warmup_sweeps=2000, thin=1, seed=0):
    rng = np.random.default_rng(seed)
    np.random.seed(rng.integers(0, 2**31-1)) # since mc.* uses global np.random

    spins = mc.init_spins(L)
    N = L * L
    E = mc.total_energy(spins, L, J)

    # warmup
    for _ in range(warmup_sweeps):
```

```

        for _ in range(N):
            x, y = np.random.randint(0, L, size=2)
            dE = mc.energy_difference(spins, L, J, x, y)
            if mc.metropolis_accept(dE, T):
                spins[x, y] *= -1
                E += dE

# sampling
samples = np.empty((n_samples, L, L), dtype=np.int8)
energies = np.empty(n_samples, dtype=np.float64)

for i in range(n_samples):
    for _ in range(thin):
        for _ in range(N):
            x, y = np.random.randint(0, L, size=2)
            dE = mc.energy_difference(spins, L, J, x, y)
            if mc.metropolis_accept(dE, T):
                spins[x, y] *= -1
                E += dE
        samples[i] = spins
        energies[i] = E

return samples, energies

def ising_stats_from_energies(E, T, Nspins):
    Emean = E.mean()
    E2mean = (E**2).mean()
    C = (E2mean - Emean**2) / (T**2)          # total C
    return (Emean / Nspins, C / Nspins)      # per spin

# =====
# 2) RBM (binary visible & hidden) + CD-k training
# spins {-1,+1} mapped to bits {0,1}
# =====
def spins_to_bits(spins_pm1):
    return ((spins_pm1 + 1) // 2).astype(np.float32)

def bits_to_spins(bits01):
    return (2*bits01 - 1).astype(np.int8)

class RBM(torch.nn.Module):
    def __init__(self, n_visible, n_hidden, seed=0):
        super().__init__()
        g = torch.Generator(device="cpu").manual_seed(seed)
        self.W = torch.nn.Parameter(0.01 * torch.randn(n_visible, n_hidden, generat
        self.a = torch.nn.Parameter(torch.zeros(n_visible)) # visible bias
        self.b = torch.nn.Parameter(torch.zeros(n_hidden))  # hidden bias

    def p_h_given_v(self, v):
        return torch.sigmoid(v @ self.W + self.b)

    def p_v_given_h(self, h):
        return torch.sigmoid(h @ self.W.t() + self.a)

    @torch.no_grad()
    def sample_bernoulli(self, p):

```

```

        return (torch.rand_like(p) < p).float()

    @torch.no_grad()
    def gibbs_step(self, v):
        ph = self.p_h_given_v(v)
        h = self.sample_bernoulli(ph)
        pv = self.p_v_given_h(h)
        v2 = self.sample_bernoulli(pv)
        return v2

    @torch.no_grad()
    def gibbs_k(self, v, k=1):
        for _ in range(k):
            v = self.gibbs_step(v)
        return v

def train_rbm_cd_k(rbm, X_bits, lr=1e-2, batch_size=100, k=5, epochs=1000, seed=0,
    """
    CD-k with the standard approximate gradient:
         $dW \sim v_0^T ph_0 - vk^T phk$ 
         $da \sim \langle v_0 - vk \rangle$ ,  $db \sim \langle ph_0 - phk \rangle$ 
    plus a simple training diagnostic: mean reconstruction error  $\|v_0 - vk\|_1$ .
    """
    rng = np.random.default_rng(seed)
    X = torch.from_numpy(X_bits).to(device)
    M = X.shape[0]
    rbm.to(device)

    for ep in range(1, epochs+1):
        perm = rng.permutation(M)
        Xp = X[perm]

        recon_err_epoch = 0.0
        nb = 0

        for start in range(0, M, batch_size):
            v0 = Xp[start:start+batch_size]
            if v0.shape[0] == 0:
                continue

            ph0 = rbm.p_h_given_v(v0)
            vk = rbm.gibbs_k(v0, k=k)
            phk = rbm.p_h_given_v(vk)

            dW = (v0.t() @ ph0 - vk.t() @ phk) / v0.shape[0]
            da = (v0 - vk).mean(dim=0)
            db = (ph0 - phk).mean(dim=0)

            rbm.W.data += lr * dW
            rbm.a.data += lr * da
            rbm.b.data += lr * db

            recon_err_epoch += (v0 - vk).abs().mean().item()
            nb += 1

        if ep % print_every == 0:

```

```

        print(f"epoch {ep:4d}/{epochs} | mean |v0-vk| = {recon_err_epoch/max(nb

    return rbm

# =====
# 3) "Generate new spin configurations randomly from each trained RBM"
#     = initialize  $v \sim \text{Bernoulli}(0.5)$  (random), then Gibbs burn-in, then sample with
# =====
@torch.no_grad()
def rbm_generate_samples(rbm, n_samples, burn_in=2000, between=10, seed=0):
    torch.manual_seed(seed)
    nV = rbm.a.numel()
    v = (torch.rand(1, nV, device=device) < 0.5).float() # random start

    # burn-in
    for _ in range(burn_in):
        v = rbm.gibbs_step(v)

    out = torch.empty(n_samples, nV, device="cpu")
    for i in range(n_samples):
        for _ in range(between):
            v = rbm.gibbs_step(v)
            out[i] = v.detach().cpu()[0]
    return out.numpy()

# =====
# 4) Run Q3: Train at  $T_1, T_2, T_3$ ; generate; compute  $\langle E \rangle$ ,  $\langle C \rangle$ ; compare to MC
# =====
L = 16
J = 1.0
Nspins = L*L

T_list = [1.00, 2.27, 3.50]

# keep MC dataset moderate so  $10^3$  epochs is practical
n_train = 10000
warmup_sweeps = 2000
thin_train = 1

# RBM hyperparams required by prompt
n_hidden = 4
lr = 1e-2 # "10^-2 or less"
k_cd = 5 # "Try  $k < 10$ "
batch_size = 100 # "100 or less"
epochs = 1000 # "aim for say  $10^3$  epochs"

# RBM generation settings
n_gen = 10000
burn_in = 2000
between = 10

results = []

for idx, T in enumerate(T_list):
    print("\n" + "="*80)
    print(f"Training RBM for T={T:.2f}")

```

```

# --- MC data (this is what you trained on in Q2, but only at these 3 temperatures)
X_spins, E_series = run_mcmc_with_warmup(
    L=L, n_samples=n_train, T=T, J=J,
    warmup_sweeps=warmup_sweeps, thin=thin_train, seed=10+idx
)
E_mc, c_mc = ising_stats_from_energies(E_series, T, Nspins)
print(f"MC baseline: <E>/N={E_mc:.6f}, c={c_mc:.6f}")

# --- RBM training ---
X_bits = spins_to_bits(X_spins).reshape(n_train, -1)
rbm = RBM(n_visible=L*L, n_hidden=n_hidden, seed=100+idx)
rbm = train_rbm_cd_k(
    rbm, X_bits,
    lr=lr, batch_size=batch_size, k=k_cd, epochs=epochs,
    seed=200+idx, print_every=100
)

# --- Generate NEW configurations from RBM (random start + Gibbs) ---
gen_bits = rbm_generate_samples(rbm, n_samples=n_gen, burn_in=burn_in, between=
gen_spins = bits_to_spins(gen_bits.reshape(n_gen, L, L))

# --- Compute <E>, <C> from generated configs using the SAME Ising energy function
# Use your mc.total_energy for consistency (loop is OK for 10k)
def energy_batch_safe(spins_pm1, J=1.0):
    s = np.asarray(spins_pm1).astype(np.int32, copy=False)

    if s.ndim == 2:
        # single config
        E = (s * np.roll(s, -1, axis=0)).sum(dtype=np.int64)
        E += (s * np.roll(s, -1, axis=1)).sum(dtype=np.int64)
        return float(-J * E)

    # batch
    E = (s * np.roll(s, -1, axis=1)).sum(axis=(1,2), dtype=np.int64)
    E += (s * np.roll(s, -1, axis=2)).sum(axis=(1,2), dtype=np.int64)
    return (-J * E).astype(np.float64)

E_gen = energy_batch_safe(gen_spins, J=J)
E_rbm, c_rbm = ising_stats_from_energies(E_gen, T, Nspins)
print(f"RBM samples: <E>/N={E_rbm:.6f}, c={c_rbm:.6f}")

results.append({
    "T": T,
    "MC <E>/N": E_mc, "MC c": c_mc,
    "RBM <E>/N": E_rbm, "RBM c": c_rbm,
    "Δ<E>/N": E_rbm - E_mc,
    "Δc": c_rbm - c_mc
})

df = pd.DataFrame(results)
print("\n=== Comparison table (RBM vs MC baseline) ===")
display(df)

```

device: cpu

Training RBM for T=1.00

MC baseline: $\langle E \rangle/N = -1.997453$, $c = 0.021227$

epoch	100/1000	mean	$v_0 - v_k$	=	0.0026
epoch	200/1000	mean	$v_0 - v_k$	=	0.0016
epoch	300/1000	mean	$v_0 - v_k$	=	0.0012
epoch	400/1000	mean	$v_0 - v_k$	=	0.0010
epoch	500/1000	mean	$v_0 - v_k$	=	0.0009
epoch	600/1000	mean	$v_0 - v_k$	=	0.0009
epoch	700/1000	mean	$v_0 - v_k$	=	0.0008
epoch	800/1000	mean	$v_0 - v_k$	=	0.0008
epoch	900/1000	mean	$v_0 - v_k$	=	0.0008
epoch	1000/1000	mean	$v_0 - v_k$	=	0.0007

RBM samples: $\langle E \rangle/N = -1.996545$, $c = 0.027926$

Training RBM for T=2.27

MC baseline: $\langle E \rangle/N = -1.467912$, $c = 1.481568$

epoch	100/1000	mean	$v_0 - v_k$	=	0.3731
epoch	200/1000	mean	$v_0 - v_k$	=	0.3819
epoch	300/1000	mean	$v_0 - v_k$	=	0.3144
epoch	400/1000	mean	$v_0 - v_k$	=	0.2888
epoch	500/1000	mean	$v_0 - v_k$	=	0.2750
epoch	600/1000	mean	$v_0 - v_k$	=	0.2612
epoch	700/1000	mean	$v_0 - v_k$	=	0.2542
epoch	800/1000	mean	$v_0 - v_k$	=	0.2615
epoch	900/1000	mean	$v_0 - v_k$	=	0.2589
epoch	1000/1000	mean	$v_0 - v_k$	=	0.2516

RBM samples: $\langle E \rangle/N = -0.737769$, $c = 0.894412$

Training RBM for T=3.50

MC baseline: $\langle E \rangle/N = -0.661433$, $c = 0.237531$

epoch	100/1000	mean	$v_0 - v_k$	=	0.4879
epoch	200/1000	mean	$v_0 - v_k$	=	0.4856
epoch	300/1000	mean	$v_0 - v_k$	=	0.4858
epoch	400/1000	mean	$v_0 - v_k$	=	0.4851
epoch	500/1000	mean	$v_0 - v_k$	=	0.4851
epoch	600/1000	mean	$v_0 - v_k$	=	0.4849
epoch	700/1000	mean	$v_0 - v_k$	=	0.4849
epoch	800/1000	mean	$v_0 - v_k$	=	0.4852
epoch	900/1000	mean	$v_0 - v_k$	=	0.4851
epoch	1000/1000	mean	$v_0 - v_k$	=	0.4856

RBM samples: $\langle E \rangle/N = -0.091166$, $c = 0.195549$

=== Comparison table (RBM vs MC baseline) ===

	T	MC $\langle E \rangle/N$	MC c	RBM $\langle E \rangle/N$	RBM c	$\Delta \langle E \rangle/N$	Δc
0	1.00	-1.997453	0.021227	-1.996545	0.027926	0.000908	0.006699
1	2.27	-1.467912	1.481568	-0.737769	0.894412	0.730144	-0.587156
2	3.50	-0.661433	0.237531	-0.091166	0.195549	0.570267	-0.041982

After training the 3 RBMs on Monte Carlo spin at $T = 1.00, 2.27, \text{ and } 3.50$, we can see that there are some differences in their respective $\langle E \rangle$ and $\langle C \rangle$ values. At $T = 1.00$, the RBM actually reproduces the Monte Carlo values very well. The discrepancies between their respective expectation values are in the order of 10^{-3} to 10^{-4} . This makes sense in low temperatures, the distribution is dominated by strongly ordered configurations. At $T = 2.27$, we get the highest discrepancies in the order of 10^{-1} . This is expected since this is the critical temperature, thus the data becomes more broad. The RBM only has 4 hidden units, so it becomes difficult to capture the variability of the data at critical temperature. At $T = 3.50$, the discrepancies lower to about an order of 10^{-1} to 10^{-2} . This could be caused by the fact that the RBM is undermodelling the remaining nearest-neighbor correlations present at $T = 3.50$. Overall, the discrepancies are mainly due to the very small hidden layer and the fact that matching critical/near-critical correlations is substantially harder than matching strongly ordered low T data.